IMPLEMENTATION DOCUMENT

Offline Business Records Application

A Windows desktop program that works without internet and without hosting, built with Nuxt 4, Vue 3 and Tailwind CSS.

Document	Implementation specification (requirements, design, build plan)
Version	1.0 - draft for approval
Date	31 October 2026
Target platform	Windows 10 and Windows 11, 64-bit laptop or desktop
Technology	Nuxt 4, Vue 3, Tailwind CSS, Electron, SQLite
Users	One person on one computer, fully offline, English and Kiswahili

Contents

1. Introduction . . . 3
2. Confirmed requirements . . . 3
3. Scope of the first release . . . 4
4. Technology stack . . . 5
5. Architecture . . . 6
6. Project structure . . . 7
7. Database design . . . 7
8. Business rules . . . 9
9. Application screens . . . 9
10. Reports . . . 12
11. Login and security . . . 13
12. Backup, restore and data safety . . . 14
13. Languages (English and Kiswahili) . . . 17
14. Internal interface (preload bridge and IPC) . . . 18
15. Packaging and installation . . . 18
16. Testing and acceptance . . . 19
17. Delivery plan . . . 20
18. Risks and how they are handled . . . 21
19. Decisions to confirm before building . . . 21
Appendix A. Database script (migration 001) . . . 23
Appendix B. Nuxt configuration and shared code . . . 25
Appendix C. Main process code patterns . . . 26
Appendix D. Build and packaging files . . . 29
Appendix E. Report queries . . . 31

1. Introduction

1.1 Purpose of this document

This document describes how to build the **Offline Business Records Application**: a Windows desktop program that lets one person record day-to-day business activity (enquiries, proforma invoices, invoices, orders, deliveries and events), find any record quickly, and produce PDF reports, Excel reports, daily summaries, monthly summaries and charts.

It is written so that a developer can build the application from it, and so that the owner can confirm the requirements before any code is written.

1.2 Can this be built with Nuxt 4, Vue 3 and Tailwind?

Yes. Nuxt is a web framework, but when it is generated as a static single-page application it needs no hosting and no server. That static application is placed inside a desktop shell (**Electron**) which gives the user a normal Windows installer, a desktop icon and a normal program window, and which stores the data in a local **SQLite** database file. The user never sees a browser, a web address, a command window or a server.

> **What the user experiences:** double-click Setup.exe, click Install, open the desktop icon, log in, and work. No internet, no hosting and no technical steps are needed at any time.

1.3 How this document is organised

- **Sections 2 to 3** - the confirmed requirements and the scope of the first release.
- **Sections 4 to 8** - technology, architecture, project structure, database and business rules.
- **Sections 9 to 13** - screens, reports, security, backup and language support.
- **Sections 14 to 19** - the internal interface, packaging, testing, delivery plan, risks and open decisions.
- **Appendices A to E** - ready-to-use database script, configuration files, code patterns and report queries.

2. Confirmed requirements

2.1 Decisions from the requirements discussion

Topic	Decision
Purpose	Record business or daily activities and report on them.
Users	One person on one computer. Reports are given to other people as PDF or Excel files.
Device	Windows laptop or desktop. No internet, server or hosting is needed after installation.
Skill level	The user has little computer experience: simple installer, large clear screens, plain language, automatic backups.
Login	Yes - a local username and password protects the application.
Numbers	The record number, proforma invoice number, invoice number, order form number and delivery form number are **typed by the user**. The application does not generate them.
Grand total	**Calculated automatically** from the item lines (quantity x unit price).
Status	The list of statuses is **editable** by the user, for example renaming "Pending" to "Confirmed", adding or hiding statuses.
Language	English and Kiswahili, switchable in Settings.
Reports	PDF reports, Excel reports, daily summaries, monthly summaries, graphs and charts.

2.2 Record fields

Field	Type	Entry	Notes
Date	Date	Required	Defaults to today; the business date of the record.
Number	Text	Typed, optional	The user's own record number. Warning if it already exists.
Client name	Text	Required	Suggestions appear from earlier records.
Contacts	Text (multi-line)	Optional	Phone numbers, e-mail addresses.
Location	Text	Optional	Suggestions from earlier records.
Source of information	Text with suggestions	Optional	For example Referral, Facebook, Walk-in. Earlier spellings are offered so reports group correctly.
Items to enquire	Item lines	Optional	Grid of item, quantity, unit price and line total.
Proforma invoice number	Text	Typed, optional	Warning if it already exists.
Invoice number	Text	Typed, optional	Warning if it already exists.
Grand total	Money	Automatic	Read-only. Sum of line totals plus tax (if tax is used).
Event date	Date	Optional	Used by the Upcoming events report and dashboard.
Order form number	Text	Typed, optional	Warning if it already exists.
Delivery form number	Text	Typed, optional	Warning if it already exists.
Status	Choice	Required	From the editable status list.
Notes	Text (multi-line)	Optional	*Extra field, not in the original list. Remove if not wanted.*

3. Scope of the first release

Included in version 1.0	Not included (possible later versions)
Local login with recovery key Record list with search, filters, sorting Record form with item lines and automatic total Editable statuses with colours Recycle bin for deleted records Dashboard with daily and monthly figures PDF and Excel reports with charts Hourly automatic backups to the computer and a Google Drive folder, restore English and Kiswahili Windows installer, desktop shortcut	Generating invoices, proforma invoices or order forms as documents Several users or several computers sharing one database Cloud storage, online sync, mobile app Sending e-mail or SMS from the application Payment tracking (part payments, balances) Stock control or accounting Automatic updates (only if the computer is online)

> **Single computer, single user.** This design intentionally has no server. If several people must later work on the same records, a different design (a shared server or a sync layer) is needed. That would be a separate project and is the main reason to confirm this choice now.

4. Technology stack

4.1 Chosen technologies

Layer	Technology	Why it is used
Interface	Vue 3 (Composition API, script setup, TypeScript)	Component-based screens and forms.
App framework	Nuxt 4 in single-page mode (ssr: false), generated as static files	File-based pages, layouts, modules; no Node server at run time.
Styling	Tailwind CSS (current stable version)	Fast, consistent, print-friendly styling.
State	Pinia	Session, settings, statuses and filter state.
Desktop shell	Electron (main process, preload script)	Installer, window, file access, printing to PDF.
Database	SQLite through better-sqlite3	One reliable local file; no database server.
PDF	Electron printToPDF on Vue print pages	Reuses the same Vue and Tailwind code as the screens.
Excel	ExcelJS	Real .xlsx files with formatting, formulas and images.
Charts	Chart.js with vue-chartjs	Works fully offline; images reusable in PDF and Excel.
Languages	@nuxtjs/i18n (vue-i18n)	English and Kiswahili text files bundled in the app.
Validation	zod	The same rules check input on screen and in the main process.
Packaging	electron-builder (NSIS)	Creates the Setup.exe installer.
Build of main process	tsup (esbuild)	Compiles the Electron TypeScript code.
Testing	Vitest (unit), optional Playwright for Electron	Calculations, database, screens.

Use the current stable version of each package when the project starts, then keep the versions fixed in package.json and the lock file so every build is repeatable.

4.2 Why Electron, and why not the alternatives

Option	Verdict	Reason
Electron + static Nuxt	**Chosen**	JavaScript end to end, mature installer tooling, simple SQLite access, built-in PDF printing.
Tauri + static Nuxt	Possible	Smaller installer, but the native side is written in Rust, and PDF printing is harder.
Browser PWA	Rejected	Data lives in browser storage and can be lost if the browser data is cleared; needs a first online visit; weak file saving.
Local Node server started by a shortcut	Rejected	Console windows, port conflicts and fragile behaviour for a non-technical user.

4.3 Rules that keep the application truly offline

- No script, font, icon or image is ever loaded from the internet. Fonts (for example Inter via @fontsource) and icons (for example lucide-vue-next) are installed as packages and bundled.
- Do not use modules that fetch icons or data from an online service at run time.
- No analytics, no update checks, no error reporting to outside servers. Google Drive backups are uploaded by the separate Google Drive for desktop program, never by this application.
- Windows requirement: Windows 10 or Windows 11, 64-bit, 4 GB RAM or more, about 400 MB free disk space for the program and early data. Current Electron versions do not run on Windows 7, 8 or 8.1.

5. Architecture

5.1 Overview

The application has two parts that run inside one Electron program. The **renderer** shows the Nuxt screens. The **main process** owns the database, the files and the reports. The renderer is never allowed to touch the database or disk directly; it can only call a fixed list of functions exposed by the preload bridge.

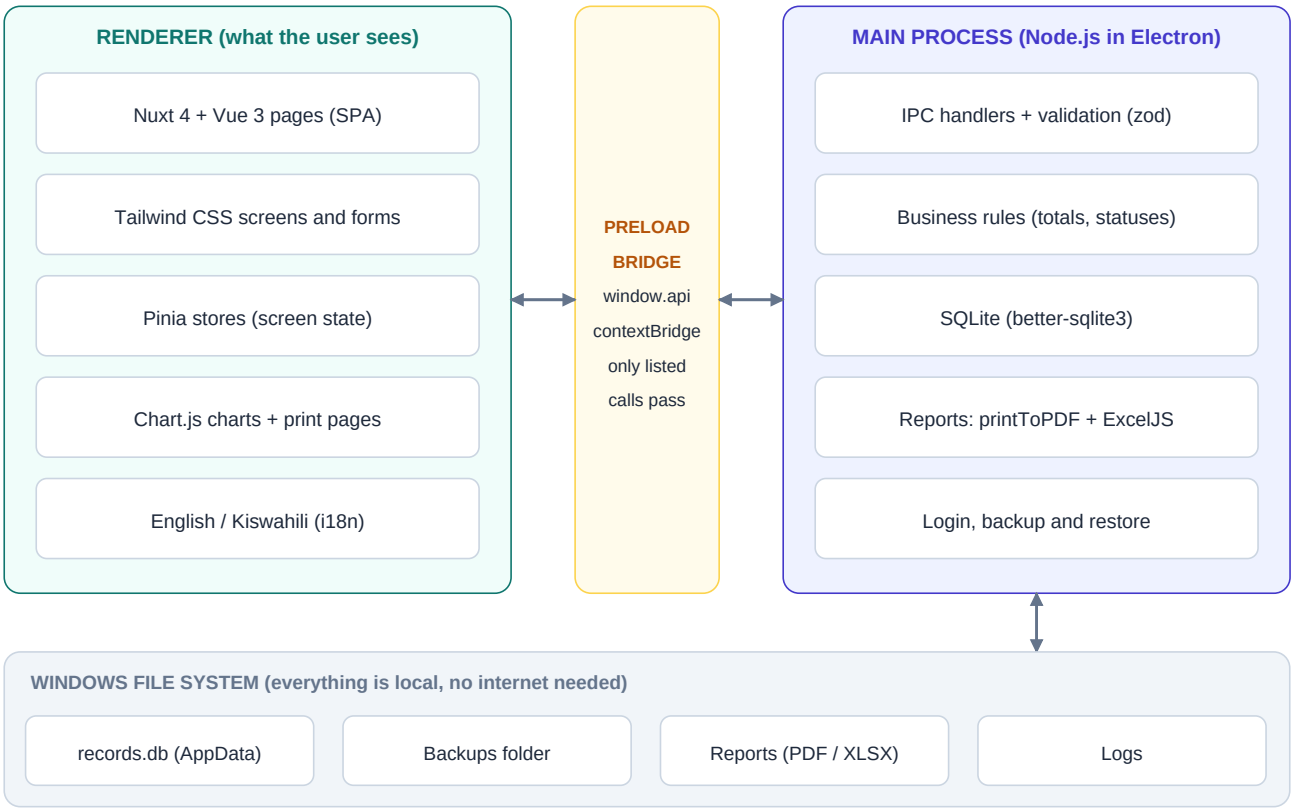


Figure 1 - Parts of the application and how they talk to each other.

5.2 How a typical action flows

1. The user clicks Save on the record form. The Vue form checks the fields and calls `window.api.records.create(...)`.
2. The preload bridge forwards the call to the main process over IPC.
3. The main process confirms the user is logged in, validates the data again with zod, and calculates the subtotal, tax and grand total itself (the screen total is only a preview).
4. Inside one SQLite transaction it saves the record and its item lines and writes an audit entry. Either everything is saved or nothing is.
5. The result returns to the screen, which shows a confirmation and refreshes the list.

5.3 Development mode and production mode

	Development	Production (installed program)
Screens	Nuxt dev server on localhost:3000 with live reload	Static files from nuxt generate, loaded from inside the installed program
Routing	Hash routing (#/records)	Hash routing (#/records) - works when loading files directly
Database	Separate development database in the user data folder	records.db in the user data folder
Start	npm run dev	Desktop icon

Two Nuxt settings make the static output work when loaded as files: `router.options.hashMode = true` and `app.baseURL = './'` (see Appendix B).

6. Project structure

```
business-records/
  app/                          Nuxt 4 source (the screens)
    pages/
      index.vue                 dashboard
      login.vue                 login
      setup.vue                 first-run wizard
      records/index.vue         record list
      records/new.vue           new record
      records/[id].vue          edit record
      records/trash.vue         recycle bin
      statuses.vue              status manager
      reports/index.vue         report chooser
      backup.vue                backup and restore
      settings.vue              settings
      help.vue                  help and about
      print/[report].vue        print-only pages used to make PDFs
    components/                 RecordTable, ItemsGrid, StatusBadge, MoneyInput, charts...
    composables/                useApi.ts, useMoney.ts, useToast.ts, useConfirm.ts
    stores/                     session.ts, settings.ts, statuses.ts
    layouts/                    default.vue (sidebar), print.vue (no chrome)
    middleware/auth.global.ts   redirects to login or setup
    assets/css/main.css         Tailwind import, fonts, print rules
  i18n/locales/en.json, sw.json
  shared/                       code used by both sides
    money.ts  currencies.ts  schemas.ts (zod)  types.ts
  electron/                     main process
    main.ts  preload.ts  ipc.ts  session.ts  auth.ts  backup.ts  logger.ts
    db/index.ts  db/migrations/001_initial.sql  db/repos/records.ts statuses.ts settings.ts
    reports/pdf.ts  reports/excel.ts  reports/queries.ts
  build/                        icon.ico, installer images
  tests/
  nuxt.config.ts  tsup.config.ts  electron-builder.yml  package.json
```

7. Database design

7.1 General decisions

- **One SQLite file**, `records.db`, in the program's user data folder (for example C:\Users\<name>\AppData\Roaming\Business Records\data).
- **Journal mode:** the default rollback journal with `synchronous = FULL`. The data file is then always complete on its own, easy to copy, and a power cut cannot corrupt it. WAL mode is not needed for one user.
- **Foreign keys on** at every connection.
- **Dates** are stored as text in the form YYYY-MM-DD (business dates) and as UTC date-time text for created and updated times. The screens show dd/mm/yyyy in local time.
- **Money is stored as whole numbers** in the smallest currency unit (TZS has no decimals, USD has two decimals) so totals never suffer rounding errors. Quantities are stored times 100 (2.5 is stored as 250).
- **Soft delete:** deleting a record sets deleted_at; it moves to the recycle bin and can be restored.
- **Schema versions** use `PRAGMA user_version`; numbered migration scripts upgrade older databases when a newer program version is installed. A backup is taken automatically before any migration.

7.2 Tables

Table	Purpose	Main columns
records	One row per business record.	record_date, record_number, client_name, contacts, location, source_of_info, proforma_no, invoice_no, event_date, order_form_no, delivery_form_no, status_id, currency, tax_rate_bp, subtotal_minor, tax_minor, grand_total_minor, notes, deleted_at
record_items	The item lines of a record.	record_id, line_no, description, quantity_x100, unit_price_minor, line_total_minor
statuses	The editable status list.	name, color, sort_order, is_default, is_active, exclude_from_totals
users	The local login.	username, display_name, password_hash, recovery_hash
settings	Key and value settings.	company details, logo path, currency, language, tax rate, backup folder, backup count, auto-lock minutes
audit_log	Who changed what and when.	at, user_id, action, entity, entity_id, details (JSON)

The complete script that creates these tables, the indexes and the starting data is in **Appendix A**.

7.3 Starting data

- **Statuses:** Pending (default), Confirmed, Proforma Sent, Invoiced, Paid, Delivered, Cancelled. "Cancelled" is marked *exclude from totals*. The user can rename, recolour, reorder, hide or add statuses at any time.
- **Settings:** currency TZS, tax rate 0 (tax off), date format DD/MM/YYYY, hourly automatic backup on (retention rules in section 12.2).

7.4 Calculation of totals

Value	Rule
Line total	round(quantity x unit price). With quantity stored times 100: round(quantity_x100 x unit_price_minor / 100).
Subtotal	Sum of all line totals of the record.
Tax	round(subtotal x tax_rate_bp / 10000). Tax rate is in basis points (1800 means 18.00 percent). The default is 0, meaning tax is not used.
Grand total	Subtotal + tax. Stored on the record every time it is saved, so reports can add it up quickly.
Where calculated	In `shared/money.ts`, used by the screen for the live preview and by the main process as the authoritative value on save.

8. Business rules

Area	Rule
Required fields	Date, client name and status. Everything else may be empty.
Manual numbers	Record, proforma, invoice, order form and delivery form numbers are free text typed by the user. The application never generates or changes them.
Duplicate numbers	When a typed number already exists on another record, the form shows a yellow warning naming the other record. The user may still save (the same number can legitimately repeat).
Items and total	The grand total cannot be typed. It follows the item lines. A record with no lines has a total of zero.
Currency	Each record has a currency (default from Settings, TZS at the start). Reports never add different currencies together; totals are shown per currency.
Renaming a status	Renaming changes the name everywhere, including in past records, because records point to the status, not to its text. A report made earlier and already handed out is not changed.
Changing a record's status	Done from the form, from the status badge in the list, or for many selected records at once.
Deleting a status	Not allowed while records use it. The user may hide it (inactive) or merge it into another status, which moves its records.
Excluded statuses	Statuses marked "exclude from totals" (for example Cancelled) are counted in the number of records but not in money totals.
Deleting a record	Moves it to the recycle bin after a confirmation. Records can be restored; permanent deletion is only from the recycle bin and needs a second confirmation.
Search	One box searches client, number, contacts, location, source, proforma, invoice, order form and delivery form, ignoring upper or lower case.
Dates	Event date may be before or after the record date. Dates are never changed by the program.
Unsaved changes	Leaving a form with unsaved changes asks for confirmation.

9. Application screens

9.1 Screen list

Screen	Purpose	Main elements
First-run setup	Runs once after installation.	Language, company details and logo, create login, show and save the recovery key, choose backup folder.
Login	Protect the data.	Username, password, "Forgot password" (uses recovery key).
Dashboard	Today and this month at a glance.	Records today and this month, value this month, upcoming events, records by status chart, last 30 days chart, backup health, quick buttons.
Records list	Find and manage records.	Search, filters, sortable table, inline status change, totals of the filtered list, export buttons.
Record form	Create or edit one record.	All fields, item grid, live totals, duplicate warnings.
Recycle bin	Recover deleted records.	Deleted list, restore, delete permanently.
Statuses	Manage the status list.	Rename, colour, reorder, hide, default, exclude from totals, merge.
Reports	Produce reports.	Report cards, parameters, on-screen preview, save as PDF or Excel.
Backup and restore	Protect against loss.	Backup now, Check my backups, copies in both folders, restore, local and Google Drive folders, set-up help.
Settings	Configure the program.	Company details, logo, language, currency, tax rate, date format, auto-lock, change password.
Help and about	Support the user.	Short how-to topics in both languages, version, data folder, contact line.

9.2 Records list

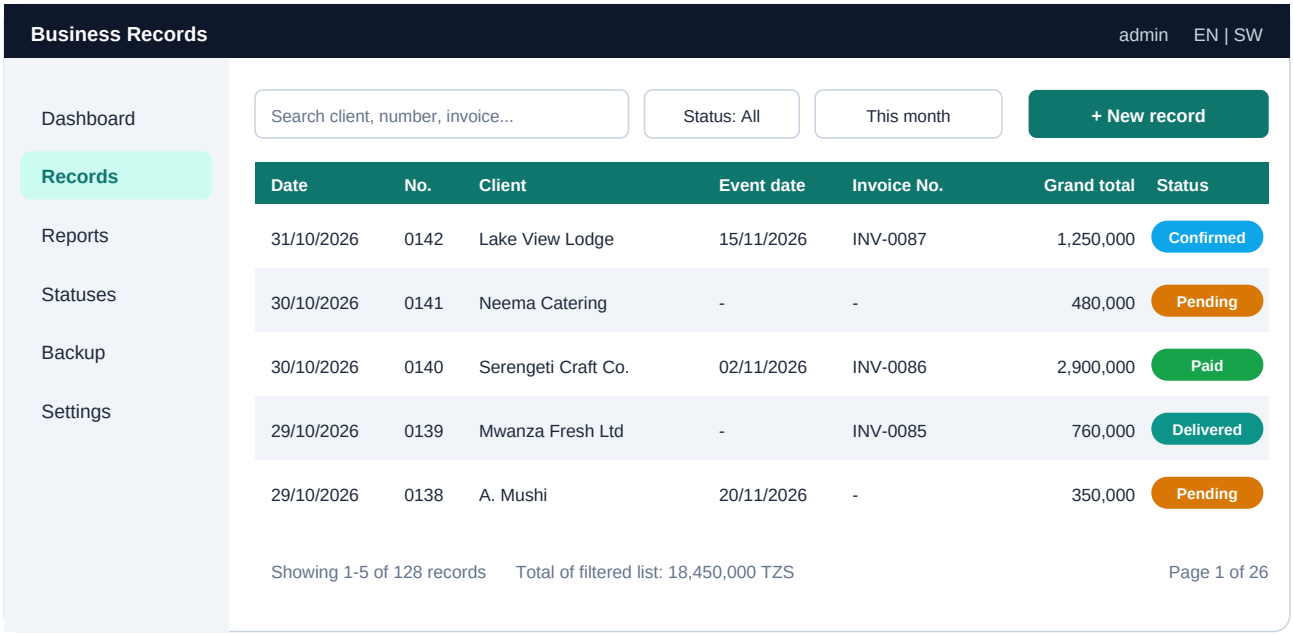


Figure 2 - Sketch of the records list (sample data is fictional).

- **Columns shown by default:** Date, No., Client, Event date, Invoice No., Grand total, Status. A column chooser adds Contacts, Location, Source, Proforma No., Order form No. and Delivery form No.
- **Filters:** date range with quick choices (Today, This week, This month, Last month, Custom), status (several at once), event date range, source of information.
- **Table:** click a heading to sort, click a row to open it, choose 25, 50 or 100 rows per page. Data is paged by the database so 100,000 records remain fast.
- **Status badge:** clicking it opens a small menu to change the status without opening the record.
- **Selecting rows** enables: change status, export selected to PDF or Excel, delete.
- **Footer:** number of records and the money total of the filtered list, shown per currency.

9.3 Record form

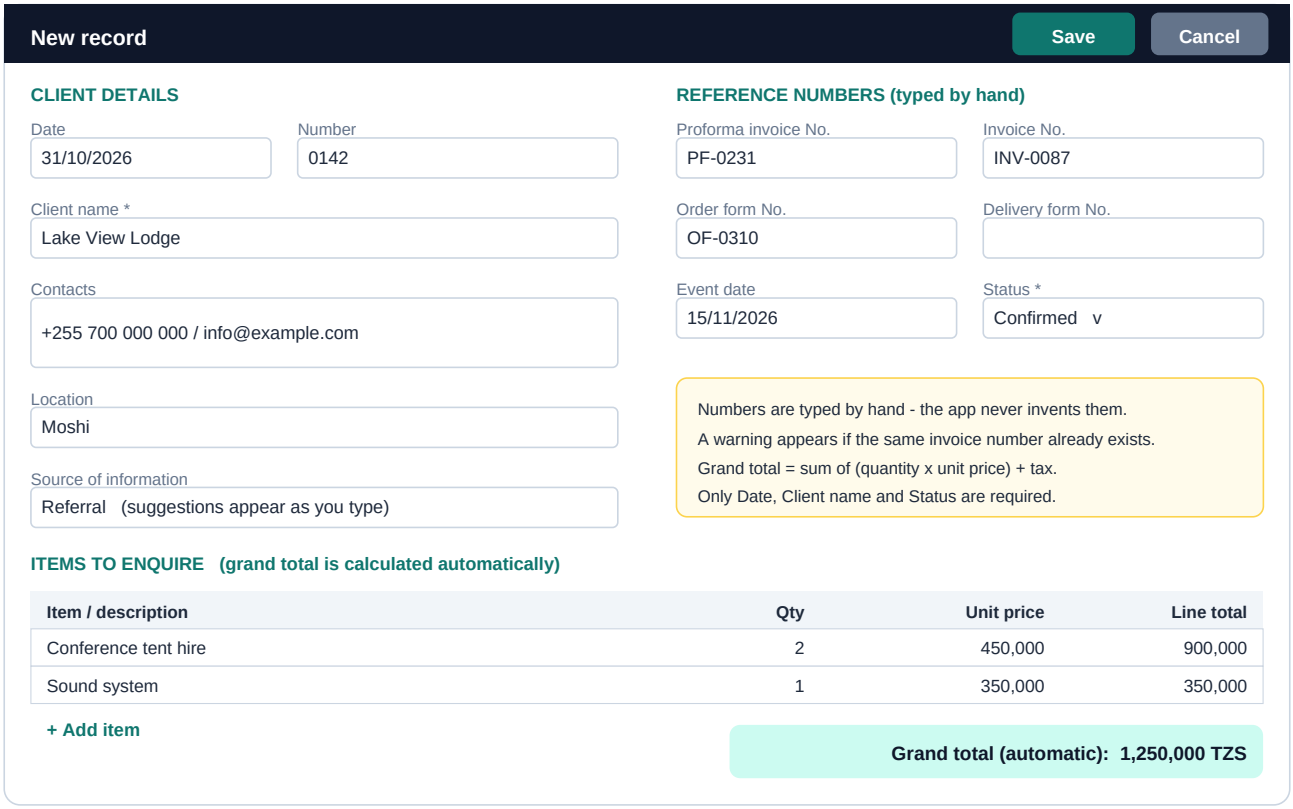


Figure 3 - Sketch of the record form.

- Date defaults to today and status defaults to the default status.
- The item grid works from the keyboard: Tab moves across, Enter adds a new line, a small button removes a line. Quantity accepts up to two decimals; prices show thousands separators as the user types.
- Subtotal, tax (if used) and grand total update instantly as lines change.
- Buttons: **Save** (Ctrl+S), **Save and add another**, **Cancel**. An existing record also shows "Created" and "Last changed" times, a **Duplicate** button and a **Delete** button.
- Error messages are in plain words in the chosen language, shown next to the field.

9.4 Dashboard

- **Cards:** records today, records this month, value this month (per currency), and the number of records in the default status (for example Pending), which usually need follow-up.
- **Upcoming events:** events in the next 7 days, soonest first, clickable.
- **Charts:** records by status (donut) and records per day for the last 30 days (bar).
- **Backup health:** "Backups OK. Last: 25 minutes ago (computer and Google Drive)" in green, or an amber or red message when a folder is missing or a copy is old (see section 12.4).
- **Quick buttons:** New record, Daily report, Monthly report, Backup now.

9.5 Statuses screen

- A list with a drag handle (reorder), a colour swatch (ten preset colours), the name (click to rename), the number of records using it, and switches for *Active* and *Exclude from totals*.
- One status is the *Default* for new records.
- A status in use cannot be deleted; the screen offers *Hide* or *Merge into...* instead.
- Colours are only an aid. Printed reports also show the status name so grey-scale printing stays clear.

9.6 Design rules for a non-technical user

- Text size of at least 16 px, large buttons, high contrast, generous spacing.
- One clear primary button per screen; everything dangerous (delete, restore a backup) asks for confirmation in plain words.
- Every list has a helpful empty state ("No records yet. Click New record to add the first one.").
- Short keyboard shortcuts for speed: Ctrl+N new record, Ctrl+F search, Ctrl+S save, Esc close.
- No technical words (database, SQL, IPC) anywhere in the interface.
- First-run wizard guides the setup; a Help page explains backup, reports and statuses in both languages.

10. Reports

10.1 Report catalogue

Report	Parameters	Contents	Output
Daily summary	Day	Number of records, money total per currency, count and value by status, list of the day's records, chart by status.	Screen, PDF, Excel
Monthly summary	Month	Key figures, table and chart per day, status donut, top 10 clients, source of information, comparison with the previous 5 months.	Screen, PDF, Excel
Period summary	From, to, optional status and client	Same layout as the monthly report for any range (a week, a quarter, a year).	Screen, PDF, Excel
Records list	Any filter of the list, chosen columns	Table of matching records with totals; landscape PDF for wide tables.	PDF, Excel
Upcoming events	Next N days or a date range	Event date, client, location, contacts, status, total.	Screen, PDF, Excel
Record sheet	One record	All fields, item lines and totals on one page.	PDF
Full data export	None	Every record and item in Excel, as an extra safety copy.	Excel

10.2 Charts

Chart	Type	Used in
Records per day	Bar	Dashboard, monthly and period summaries
Value per day	Bar or line (per currency)	Monthly and period summaries
Records by status	Donut	Dashboard, daily, monthly and period summaries
Top clients by value	Horizontal bar	Monthly and period summaries
Source of information	Bar	Monthly and period summaries
Last 6 months comparison	Bar	Monthly summary

10.3 How PDF reports are made

Each report has a **print page** in the Nuxt project (for example `/print/monthly`). It is ordinary Vue and Tailwind code that shows the report on screen as a preview. To make the PDF, the main process opens that page in a hidden window, waits until the page announces it is ready (data loaded and charts drawn), and calls Electron's `printToPDF`. The result is saved through a Save dialog.

- **Same code, same look:** the preview on screen and the PDF always match, and Swahili or English text, charts and tables need no separate PDF design.
- **Page layout:** A4, portrait by default (landscape for wide tables). Header with logo, company name, address and phone; report title and period; "Generated on dd/mm/yyyy hh:mm by user". Footer with "Page x of y".

- **Long tables** repeat their heading on every page and never split a row across pages (CSS print rules).
- **Charts** are drawn with animation turned off at a fixed size so they print sharply.
- **Fonts** are bundled, so PDFs look the same on every computer.
- A pure-code alternative (pdfmake) is possible, but it needs a second layout design for every report. It is kept as a fallback only.

10.4 How Excel reports are made

- Made in the main process with ExcelJS as real .xlsx files that open in Excel and LibreOffice.
- **Sheet "Summary":** key figures, counts and values by status, per-day table, with chart images placed beside the tables.
- **Sheet "Records":** one row per record, all columns, bold frozen heading row, filter buttons, dates as real dates (dd/mm/yyyy), money as real numbers with thousands separators.
- **Sheet "Items":** one row per item line with the record number and client.
- **Totals use formulas** (SUBTOTAL, SUMIF per currency) so they stay correct when the person filters the sheet.
- ExcelJS cannot create native Excel charts, so charts are inserted as images captured from Chart.js.

10.5 Saving and sharing

- A report is shown on screen first. **Save as PDF** and **Save as Excel** open a Save dialog that starts in Documents\Business Records\Reports with a clear file name such as `Monthly-Summary_2026-10.pdf`.
- After saving, the program offers **Show in folder** and **Open file**.
- **A copy also goes to Google Drive.** Every saved report is also copied into the Reports sub-folder of the Google Drive folder (see section 12.3), where Google Drive for desktop uploads it. If the Drive folder is not available, the report is still saved in Documents, a notice is shown, and the copy is retried the next time the program starts.
- **Automatic daily report (optional setting, off by default):** when the program is closed, today's daily summary is saved as a PDF in the same two places without the user pressing anything.
- **Print** sends the same page to the Windows printer dialog.
- The files are ordinary files: the user can copy them to a USB drive, attach them to e-mail or share them with any program they already use.

11. Login and security

11.1 Login

- The first-run wizard creates the one login (username and password, minimum 6 characters).
- Passwords are never stored. Only a salted hash (scrypt from Node's built-in crypto library) is kept.
- After five wrong attempts the login waits 30 seconds before the next try.
- Optional **auto-lock** after a chosen number of idle minutes returns to the login screen.
- The user can change the password in Settings.

11.2 Recovery key

Because the user may forget the password, the wizard creates a **recovery key** (16 characters in groups of four, using letters and digits that cannot be confused). It is shown once, with buttons to **print** or **save it as a PDF**, and the user must tick "I have saved it" before continuing. Only a hash of it is stored. The "Forgot password" link accepts the recovery key and lets the user choose a new password (and receive a new recovery key).

> **Honest limit:** the login protects the program, not the data file. Someone who can open the computer's files could read the database with a database tool. If the records are sensitive, full-file encryption (an encrypted SQLite build) and Windows disk encryption (BitLocker) should be added; this is listed as an open decision in section 19.

11.3 Electron hardening checklist

- contextIsolation on, nodeIntegration off, sandbox on; the renderer has no direct Node or file access.
- The preload script exposes only named functions; every IPC channel validates its input with zod and checks that the user is logged in (except the login and setup channels).
- Window opening and navigation to outside addresses are blocked; the program never loads remote content.
- A Content Security Policy limits the page to its own files. Nuxt adds one small inline start-up script, so the policy must allow inline scripts (`'unsafe-inline'`) or use a hash; this is acceptable because no remote content is ever loaded.
- Only one copy of the program can run at a time (single-instance lock), so two copies never write to the same database.
- Errors are logged to a local log file; no personal data is sent anywhere.

12. Backup, restore and data safety

No design can make data loss impossible, but copies kept in **several places**, made **automatically**, make it very unlikely. The application uses three layers. The third layer, a copy on **Google Drive**, is what protects the records if the laptop is damaged, stolen or infected.

12.1 The three layers

Layer	What it protects against	How it works
1. Safe saving	Power cuts, crashes, half-saved records, deleting by mistake.	Every save is all-or-nothing (one SQLite transaction). Deleted records go to the recycle bin. The database is set to write fully to disk (`synchronous = FULL`).
2. Backups on the computer	A damaged database file, a bad upgrade, a wrong restore.	Automatic backup copies in the program's own data folder, away from the Documents folder (see 12.2).
3. Copy on Google Drive	Laptop failure, theft, fire, virus, a damaged disk.	The same backup files, and every saved report, are also written to a folder that Google Drive for desktop uploads to the user's Google account (see 12.3).

12.2 Backup behaviour

Feature	Behaviour
When backups are made	(1) when the program starts, (2) every hour while the program is open *and data has changed since the last backup*, (3) when the program is closed, (4) before any database upgrade, and (5) on demand with **Backup now**.
Maximum work at risk	About one hour of typing, because of the hourly backup (and only if the computer itself is lost between two backups).
File names	`records_2026-11-01_1600.db`. Upgrade copies are named `before-migration`.
Local backup folder	AppData\Roaming\Business Records\backups, inside the program's own data area and **not** in Documents, so backup files never mix with the reports. Always on; it cannot be turned off. The user never needs to open it: the copies are listed on the Backup screen.
Google Drive backup folder	A second destination: the Backups sub-folder of the Google Drive folder chosen in Settings (see 12.3). Every backup is copied to both places.
How long copies are kept	Hourly copies for the last 2 days, then one copy per day for 30 days, then one copy per month for 12 months. Older copies are removed automatically. "Before-migration" copies are kept until the user deletes them.
How the copy is made	SQLite `VACUUM INTO` (or the better-sqlite3 backup function) writes a consistent, compact copy while the program is running. The copy is written under a temporary name and renamed when complete, so Google Drive never uploads half a file.
Checking each copy	After writing, the program opens the copy, runs an integrity check and compares the record count. A bad copy is deleted and the user is warned.
Weekly Excel export	Once a week the program also saves the "Full data export" Excel file in the Reports folder in Documents and in Google Drive. It can be opened without the program, as a last-resort copy.
Restore	The user picks a backup from the list (or a file). The program checks the file (integrity check, expected tables, compatible version), makes a safety copy of the current data, replaces the database and restarts. A backup from a newer program version is refused.
Check my backups	A button that opens the newest backup in each folder, runs the same health checks and shows "Local backup OK, 1 hour ago" and "Google Drive folder OK, 1 hour ago".
Uninstalling	Removing the program keeps the data and backups; they are not deleted by the uninstaller.

12.3 Google Drive backup

The program does **not** talk to Google itself. It only writes backup files into a normal Windows folder, and the free **Google Drive for desktop** program uploads that folder to the user's Google account in the background whenever the computer is online. This keeps the application fully offline, needs no Google sign-in or password inside the application, and keeps working during internet outages: the files simply upload later. The same Google Drive folder also receives a copy of every saved report, so the reports are safe off the computer as well.

One-time set-up (done once during installation, by the developer or the owner)

1. Install Google Drive for desktop on the Windows computer and sign in with the Google account that will hold the backups (a dedicated business account is best).
2. In File Explorer, open the Google Drive location and create a folder named **Business Records**.
3. In the application's first-run wizard (or Settings, Backup), choose that folder as the **Google Drive folder**. The program creates two sub-folders in it, **Backups** and **Reports**, and writes a small test file to confirm the folder works.
4. Check on drive.google.com (any computer) that the two sub-folders and the test file appear. Then press Backup now and save one report, and check that both files arrive.
5. Turn on 2-Step Verification for that Google account and write down the account recovery details, so the backups themselves cannot be locked away from the owner.

Rules that keep this safe

- **Never put the live database (records.db) inside the Google Drive folder.** Sync tools can upload a file halfway through a write, or create "conflict" copies, and damage it. Only finished backup copies and saved reports go to Google Drive. The live database always stays in the program's own data folder.
- Use a folder inside **My Drive** (the synced Google Drive location), not a folder on a shared drive that other people can edit or delete.
- Both Drive modes work: "Stream files" keeps files online and "Mirror files" also keeps a copy on the computer. Either way the upload happens in the background.
- Backups and reports contain business data, so the Google account should have a strong password and 2-Step Verification, and the Business Records folder must not be shared with other people unless the owner wants them to see the reports.
- Google Drive storage is limited by the account's free quota (15 GB, shared with Gmail and Photos). A record database is small, so the retention rules above keep the folder small even with many years of records. The program warns if the folder cannot be written to.

What the application can and cannot check

The application can	The application cannot
Confirm that the Google Drive folder exists and can be written to. Confirm that the newest backup was written there and is healthy. Warn if the folder is missing or read-only, for example when the Drive program is not running. Warn if no backup reached the folder for 2 days.	See whether Google Drive has **finished uploading**. That is known only to Google Drive for desktop. Warn about internet problems on its own. Know if the user signed out of Google Drive for desktop.

Because of the last limit, the user guide asks the user to look at the Google Drive icon in the Windows tray once a week (it shows a check mark when everything is uploaded), and the dashboard shows the time of the newest backup in each folder. Once a month, the owner should open drive.google.com and confirm that recent backup files are there.

12.4 Warnings the user sees

Situation	Message on the dashboard
Everything fine	Green: "Backups OK. Last: 25 minutes ago (computer and Google Drive)."
No backup in the local folder for 2 days	Red: "No backup for 2 days. Press Backup now."
Google Drive folder missing or unreachable	Red: "Google Drive folder not found. Open Google Drive for desktop, then press Check my backups."
Google Drive copy older than 2 days, local copy fine	Amber: "Google Drive copy is old. Open Google Drive for desktop and check that it is signed in."
Google Drive not set up yet	Amber until set up: "Your data is only on this computer. Set up the Google Drive backup."
A report could not be copied to Google Drive	Amber notice after saving: "Report saved on this computer. The Google Drive folder was not found; the copy will be made later."
Backup copy failed its health check	Red: "A backup was damaged and was removed. Press Backup now. If it happens again, contact support."

> **Weekly habit for the user (one line in the user guide):** look at the dashboard. If the backup line is green, nothing else is needed. If it is amber or red, follow the message on the screen.

12.5 Where every file is stored

What	Where	Purpose
Live records database	AppData\Roaming\Business Records\data\records.db	The working data. Never copied by hand, never placed in a synced folder.
Database backups (local)	AppData\Roaming\Business Records\backups	Restore after a mistake or a damaged file. Not in Documents.
Saved reports (PDF, Excel)	Documents\Business Records\Reports	The files the user opens, prints and gives to other people.
Google Drive: Backups	Google Drive\Business Records\Backups	Database backups and the weekly Excel export, safe if the computer is lost.
Google Drive: Reports	Google Drive\Business Records\Reports	A copy of every saved report, reachable from any device the owner signs in on.

Reports cannot restore the application. A PDF or Excel report is for reading and sharing; only a database backup can bring back every record, item and status. That is why both kinds of file are kept.

13. Languages (English and Kiswahili)

- All interface text lives in two files, `en.json` and `sw.json`, bundled into the program. A language switch in the top bar and in Settings changes the language immediately and remembers the choice.
- Dates, numbers and money are formatted with the chosen language (en-TZ or sw-TZ conventions).
- Reports follow the language chosen at the time they are created.
- Status names and company details are typed by the user and are not translated.
- A native Kiswahili speaker should review the final wording before release.

13.1 Starting translations of the main labels (to be reviewed)

English	Kiswahili
Date	Tarehe
Number	Namba
Client name	Jina la mteja
Contacts	Mawasiliano
Location	Mahali
Source of information	Chanzo cha taarifa
Items to enquire	Bidhaa au huduma za kuulizia
Proforma invoice number	Namba ya ankara ya awali (proforma)
Invoice number	Namba ya ankara
Grand total	Jumla kuu
Event date	Tarehe ya tukio
Order form number	Namba ya fomu ya oda
Delivery form number	Namba ya fomu ya uwasilishaji
Status	Hali
Daily summary	Muhtasari wa siku
Monthly summary	Muhtasari wa mwezi
Backup	Nakala rudufu

14. Internal interface (preload bridge and IPC)

The renderer calls `window.api.<group>.<function>(input)`. Every call returns `{ ok: true, data }` or `{ ok: false, error: { code, message } }`. A small `useApi()` helper unwraps the result and shows a friendly message on error. The channels are:

Channel	Input	Returns
auth:status	-	hasUser, loggedIn, user
auth:setup	username, displayName, password, company details	recoveryKey
auth:login / logout	username, password / -	user / -
auth:changePassword	oldPassword, newPassword	-
auth:recover	username, recoveryKey, newPassword	newRecoveryKey
records:list	search, dateFrom, dateTo, statusIds, eventFrom, eventTo, source, sort, page, pageSize	rows, total, sums per currency
records:get	id	record, items
records:create / update	record, items (and id)	id
records:setStatus	ids, statusId	changed count
records:delete / restore / purge	ids	changed count
records:duplicate	id	new id
records:checkDuplicates	field, value, excludeId	matching records
records:suggestions	field (client, location, source)	list of text
statuses:list / create / update / reorder / delete / merge	varies	statuses, usage counts
settings:get / set / chooseLogo	key and value / -	settings
reports:data	type, params	JSON used by report and print pages
reports:exportPdf / exportExcel	type, params, suggestedName (+ chart images for Excel)	saved path, Drive copy result
backup:now / list / restore	varies	file name, list, status
backup:chooseFolder / testFolder	local or Google Drive folder	folder, write test result
backup:check	-	health of the newest copy in each folder
system:openFolder / info	path / -	version, data folder

15. Packaging and installation

15.1 Installer

- Built with electron-builder as an **NSIS installer**: `BusinessRecords-Setup-1.0.0.exe`, roughly 80 to 120 MB, about 300 MB when installed.
- **Per-user installation** (no administrator rights needed), with a choice of folder, a desktop shortcut and a Start menu entry, and an uninstaller.
- Program icon (.ico, 256 px) and application name chosen with the owner.
- Uninstalling does not delete the user's data or backups.
- The native database module (better-sqlite3) is rebuilt for Electron during the build and unpacked from the program archive (see Appendix D).
- Build the installer on a Windows machine, then test it on a **clean Windows 10 computer that has no development tools**.

15.2 Code signing and the Windows warning

An installer without a digital signature makes Windows SmartScreen show "Windows protected your PC / Unknown publisher". The user can still install it by clicking **More info** and then **Run anyway**; the user guide must show this with a screenshot. To remove the warning, buy a code-signing certificate from a certificate authority and configure electron-builder to sign the installer. This is a business decision (see section 19).

15.3 Updating

- Without internet: send the new Setup.exe; installing over the old version keeps all data. On first start the program takes a backup and upgrades the database if needed.
- With internet (optional, later): electron-updater can download updates automatically; it is not part of version 1.0 because the application must work fully offline.
- An optional **portable version** (a single .exe that runs from a USB drive) can also be built if the owner wants it, but the installed version is recommended because of backups and shortcuts.

15.4 First-run wizard

1. Choose language (English or Kiswahili).
2. Enter company name, address, phone, e-mail, logo (optional) and the main currency.
3. Create the username and password.
4. Show the recovery key; print or save it; tick "I have saved it".
5. Backups and reports: the local backup folder and the Reports folder in Documents are set automatically. Choose the Google Drive folder (the program creates the Backups and Reports sub-folders and tests it) or choose Later; the dashboard stays amber until it is set. Press Finish.

16. Testing and acceptance

16.1 Test plan

Area	What is tested
Calculations	Line totals, subtotal, tax and grand total with decimals, large numbers and several currencies (automatic unit tests).
Database	Create, edit, delete, restore; transactions; migrations from every older version; foreign keys; duplicate warnings (tests against a temporary database).
Screens	Required-field messages, keyboard use of the item grid, filters, sorting, status changes, unsaved-changes warning, language switch.
Reports	Every report in PDF and Excel with 0, 1 and 5,000 records; Swahili and English; page breaks; totals match the screen; the Excel file opens in Excel and LibreOffice.
Backup and restore	Hourly, start-up and close backups appear in both folders and rotate by the retention rules; no backup is made when nothing changed; restore replaces data correctly; a damaged file is rejected; a missing or read-only Google Drive folder shows the right warning; the live database is never written into the Drive folder.
Google Drive end to end	On a real Windows computer with Google Drive for desktop: backup files appear on drive.google.com; backups made while offline upload after the internet returns; saved reports and the weekly Excel export arrive in the Reports folder; a backup downloaded from Drive on another computer restores correctly.
Security	Wrong password handling, recovery key, auto-lock, renderer cannot reach Node or the disk.
Installation	Clean Windows 10 and Windows 11 machines; install, run, uninstall; reinstall keeps data; upgrade from 1.0 to a test 1.1 keeps data; SmartScreen steps documented.
Reliability	Power-cut simulation during a save; closing the program during a backup.
Performance (targets)	Start in under 5 seconds; list opens in under 1 second with 50,000 records; saving under 0.3 seconds; monthly PDF in under 10 seconds.

16.2 Acceptance checklist

- The installer runs on a clean Windows 10 computer without internet and without administrator rights.
- The first-run wizard completes and the recovery key can be printed or saved.
- A record with all 14 fields can be created, edited, found by search, and deleted and restored.
- The grand total is always correct and cannot be typed over.
- Typed numbers are never changed or generated by the program.
- A status can be renamed (Pending to Confirmed) and the change shows in the list, dashboard and reports.
- Daily and monthly summaries, record lists and upcoming events can be produced as PDF and Excel with charts, in both languages.
- Every saved report appears in Documents\Business Records\Reports and in the Google Drive Reports folder.
- Backups reach both the local folder and the Google Drive folder, a backup downloaded from Google Drive restores on another computer, the backup warnings work, and uninstalling keeps data.
- The program itself makes no network connection (Google Drive for desktop does the uploading).

17. Delivery plan

Effort is a rough estimate for one developer who already knows Nuxt, Vue and Tailwind.

Phase	Work	Result	Days
0. Proof of concept	Nuxt + Electron + better-sqlite3 "hello" window, build the installer, install on a clean Windows 10 computer, print one test PDF.	Confirms the toolchain and removes the main risks before real work	2
1. Foundation	Database and migrations, IPC pattern, login and recovery key, settings, i18n set-up, layout and navigation.	Application shell that starts, logs in and stores settings	4
2. Records	Record list with search and filters, record form, item grid and totals, statuses screen, duplicates warnings, recycle bin.	Complete data entry and management	6
3. Dashboard	Dashboard cards, upcoming events, charts, backup health.	Daily working screen	3
4. Reports	Print pages, PDF engine, Excel engine, report screens, charts, save dialogs, copy to Google Drive, optional automatic daily report.	All reports in PDF and Excel, saved in two places	7
5. Safety and onboarding	Backups (hourly, retention, health checks, Google Drive folder, warnings), restore, weekly Excel export, first-run wizard, help page, change password, auto-lock.	Safe for a non-technical user	5
6. Finish	Kiswahili review, test plan, performance checks, installer polish, user guide (English and Kiswahili), handover.	Release candidate and documents	4

Total about 31 working days (six weeks) including testing. Phase 0 should be done and shown to the owner before the rest is started.

17.1 Deliverables

- Windows installer (Setup.exe) and, optionally, a portable build.
- Complete source code with a README explaining how to build and package.
- Database description (data dictionary) and migration scripts.
- User guide in English and Kiswahili (short, with screenshots), including backup and SmartScreen steps.
- Test results and the acceptance checklist signed off.

18. Risks and how they are handled

Risk	Handling
The native database module fails to build for Electron	Do Phase 0 first; use matching prebuilt binaries; build on Windows; unpack the module from the archive.
The user loses data (disk failure, laptop stolen, deleted by mistake)	Hourly automatic backups to the computer and to Google Drive, recycle bin, backup warnings, weekly Excel export.
Google Drive stops uploading (signed out, full quota, program not running)	Dashboard warns when the Drive folder is missing or its newest copy is old; weekly tray-icon check; monthly check on drive.google.com; local copies continue meanwhile.
Sync damages the database	Only finished backup copies go to Google Drive; the live database is never placed in a synced folder.
The password and recovery key are both lost	Recovery key must be saved before continuing; document a developer-only reset procedure that needs physical access to the computer.
Windows SmartScreen frightens the user	Code signing certificate, or a clear screenshot guide.
PDF pages break badly with long reports	Print CSS rules, test with 5,000 records, landscape option.
Antivirus flags the installer	Sign the installer; keep dependencies few and up to date.
Old Windows version	State the minimum (Windows 10, 64-bit) before the project starts.
Wrong Kiswahili wording	Review by a native speaker; labels are in one file and easy to change.
Requirements grow (several users, invoices)	Kept out of version 1.0 (section 3); database is versioned so features can be added safely.
Security updates for Electron	Plan a rebuild and a new installer once or twice a year.

19. Decisions to confirm before building

The design above uses the recommended choice for each item. Please confirm or change them.

#	Question	Recommended
1	Items to enquire: is it one grid of priced lines (assumed), or a free text plus a separate price list?	One grid of lines; grand total is its sum.
2	Currency: only TZS, or also USD and others?	Per-record currency, default TZS; totals per currency.
3	Tax: not used, or one rate such as VAT?	Optional single rate in Settings, off by default.
4	Should Cancelled records be left out of money totals?	Yes, using the "exclude from totals" switch on a status.
5	Is the record Number required or unique?	Optional, warn on duplicates, never block.
6	Report appearance: company logo and details at the top of every report?	Yes, from Settings.
7	Application name and icon?	Owner to provide; working name "Business Records".
8	Must the database file itself be encrypted?	Not in 1.0; add Windows BitLocker advice. Add encryption if the data is sensitive.
9	Code signing certificate for the installer?	Yes if many users will install it; otherwise use the screenshot guide.
10	Extra Notes field on each record?	Include (can be removed).
11	Which Google account will hold the backups?	A dedicated business account with 2-Step Verification; the owner keeps the recovery details.
12	Who installs and signs in to Google Drive for desktop?	Developer, at installation time, together with the owner.

#	Question	Recommended
13	Should the daily report also be e-mailed to a Gmail address?	Not decided. Not part of this document yet; can be added as a separate section (about 3 to 4 days).
14	Save the daily report automatically when the program is closed?	Optional setting, off by default; saved in Documents and Google Drive.

Appendix A. Database script (migration 001)

File `electron/db/migrations/001_initial.sql`. The program sets `foreign_keys = ON` and `synchronous = FULL` when it opens the database.

electron/db/migrations/001_initial.sql

```
CREATE TABLE users (
  id             INTEGER PRIMARY KEY AUTOINCREMENT,
  username       TEXT NOT NULL UNIQUE,
  display_name   TEXT NOT NULL,
  password_hash  TEXT NOT NULL,                -- scrypt, "salt:hash" in hex
  recovery_hash  TEXT NOT NULL,                -- hash of the recovery key
  created_at     TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at     TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE statuses (
  id                 INTEGER PRIMARY KEY AUTOINCREMENT,
  name               TEXT NOT NULL UNIQUE COLLATE NOCASE,
  color              TEXT NOT NULL DEFAULT '#64748b',
  sort_order         INTEGER NOT NULL DEFAULT 0,
  is_default         INTEGER NOT NULL DEFAULT 0,   -- used for new records
  is_active          INTEGER NOT NULL DEFAULT 1,   -- 0 = hidden from the dropdown
  exclude_from_totals INTEGER NOT NULL DEFAULT 0    -- e.g. Cancelled
);

CREATE TABLE records (
  id                 INTEGER PRIMARY KEY AUTOINCREMENT,
  record_date        TEXT NOT NULL,                -- YYYY-MM-DD
  record_number      TEXT,                         -- typed by the user
  client_name        TEXT NOT NULL,
  contacts           TEXT,
  location           TEXT,
  source_of_info     TEXT,
  proforma_no        TEXT,                         -- typed by the user
  invoice_no         TEXT,                         -- typed by the user
  event_date         TEXT,                         -- YYYY-MM-DD or NULL
  order_form_no      TEXT,                         -- typed by the user
  delivery_form_no   TEXT,                         -- typed by the user
  status_id          INTEGER NOT NULL REFERENCES statuses(id),
  currency           TEXT NOT NULL DEFAULT 'TZS',
  tax_rate_bp        INTEGER NOT NULL DEFAULT 0,    -- 1800 = 18.00 percent
  subtotal_minor     INTEGER NOT NULL DEFAULT 0,
  tax_minor          INTEGER NOT NULL DEFAULT 0,
  grand_total_minor  INTEGER NOT NULL DEFAULT 0,    -- calculated on every save
  notes              TEXT,
  created_at         TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at         TEXT NOT NULL DEFAULT (datetime('now')),
  deleted_at         TEXT                          -- NULL = active, else in recycle bin
);

CREATE TABLE record_items (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  record_id         INTEGER NOT NULL REFERENCES records(id) ON DELETE CASCADE,
  line_no           INTEGER NOT NULL,
  description       TEXT NOT NULL,
  quantity_x100     INTEGER NOT NULL DEFAULT 100,   -- 2.5 is stored as 250
  unit_price_minor  INTEGER NOT NULL DEFAULT 0,
  line_total_minor  INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE settings (
  key    TEXT PRIMARY KEY,
  value  TEXT NOT NULL
);

CREATE TABLE audit_log (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  at         TEXT NOT NULL DEFAULT (datetime('now')),
  user_id    INTEGER REFERENCES users(id),
  action     TEXT NOT NULL,                        -- record.create, status.rename, ...
  entity     TEXT NOT NULL,
  entity_id  INTEGER,
  details    TEXT                                  -- JSON with old and new values
);

CREATE INDEX idx_records_date     ON records(record_date) WHERE deleted_at IS NULL;
CREATE INDEX idx_records_status   ON records(status_id);
CREATE INDEX idx_records_client   ON records(client_name COLLATE NOCASE);
CREATE INDEX idx_records_event    ON records(event_date);
CREATE INDEX idx_records_invoice  ON records(invoice_no);
CREATE INDEX idx_records_proforma ON records(proforma_no);
CREATE INDEX idx_items_record     ON record_items(record_id);
```

```
INSERT INTO statuses (name, color, sort_order, is_default, exclude_from_totals) VALUES
  ('Pending',       '#d97706', 1, 1, 0),
  ('Confirmed',     '#0ea5e9', 2, 0, 0),
  ('Proforma Sent', '#8b5cf6', 3, 0, 0),
  ('Invoiced',      '#6366f1', 4, 0, 0),
  ('Paid',          '#16a34a', 5, 0, 0),
  ('Delivered',     '#0d9488', 6, 0, 0),
  ('Cancelled',     '#dc2626', 7, 0, 1);

INSERT INTO settings (key, value) VALUES
  ('company_name', ''), ('company_address', ''), ('company_phone', ''), ('company_email', ''),
  ('logo_path', ''), ('currency', 'TZS'), ('tax_rate_bp', '0'), ('language', 'en'),
  ('date_format', 'DD/MM/YYYY'), ('backup_dir', ''), ('backup_dir_drive', ''), ('auto_backup', '1'),
  ('autolock_minutes', '0'), ('last_backup_local', ''), ('last_backup_drive', ''),
  ('reports_dir', ''), ('copy_reports_to_drive', '1'), ('auto_daily_report', '0');
```

Appendix B. Nuxt configuration and shared code

nuxt.config.ts

```
// nuxt.config.ts
import tailwindcss from '@tailwindcss/vite'

export default defineNuxtConfig({
  compatibilityDate: '2025-07-15',
  ssr: false,                               // single-page app: no server needed
  devtools: { enabled: false },
  telemetry: false,
  modules: ['@pinia/nuxt', '@nuxtjs/i18n'],
  css: ['~/assets/css/main.css'],           // @import "tailwindcss"; fonts; print rules
  vite: { plugins: [tailwindcss()] },
  app: {
    baseURL: './',                          // relative paths so files load from disk
    buildAssetsDir: 'assets/',
    head: {
      title: 'Business Records',
      meta: [{ 'http-equiv': 'Content-Security-Policy',
               content: "default-src 'self'; script-src 'self' 'unsafe-inline'; " +
                        "style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; " +
                        "font-src 'self' data:" }],
    },
  },
  router: { options: { hashMode: true } },  // #/records instead of /records
  i18n: {
    strategy: 'no_prefix',
    defaultLocale: 'en',
    detectBrowserLanguage: false,
    locales: [
      { code: 'en', name: 'English',   file: 'en.json' },
      { code: 'sw', name: 'Kiswahili', file: 'sw.json' },
    ],
    // locale files are bundled into the app; check the module documentation for the
    // exact langDir location in the Nuxt 4 folder layout
  },
  nitro: { preset: 'static' },
})
```

shared/currencies.ts and shared/money.ts

```
// shared/currencies.ts
export const CURRENCIES: Record<string, { decimals: number; name: string }> = {
  TZS: { decimals: 0, name: 'Tanzanian shilling' },
  USD: { decimals: 2, name: 'US dollar' },
  EUR: { decimals: 2, name: 'Euro' },
  KES: { decimals: 2, name: 'Kenyan shilling' },
}

// shared/money.ts
import { CURRENCIES } from './currencies'

export interface ItemInput { quantityX100: number; unitPriceMinor: number }

export function lineTotal(i: ItemInput): number {
  return Math.round((i.quantityX100 * i.unitPriceMinor) / 100)
}

export function totals(items: ItemInput[], taxRateBp: number) {
  const subtotal = items.reduce((sum, i) => sum + lineTotal(i), 0)
  const tax = Math.round((subtotal * taxRateBp) / 10000)
  return { subtotal, tax, grandTotal: subtotal + tax }
}

export function formatMoney(minor: number, currency: string, locale: string): string {
  const decimals = CURRENCIES[currency]?.decimals ?? 2
  const nf = new Intl.NumberFormat(locale, {
    minimumFractionDigits: decimals, maximumFractionDigits: decimals,
  })
  return `${nf.format(minor / 10 ** decimals)} ${currency}`
}
```

Appendix C. Main process code patterns

electron/main.ts

```
// electron/main.ts
import { app, BrowserWindow } from 'electron'
import path from 'node:path'
import { openDatabase } from './db'
import { registerIpc } from './ipc'
import { runExitBackup } from './backup'

const isDev = !app.isPackaged
if (!app.requestSingleInstanceLock()) app.quit()      // only one copy at a time

let win: BrowserWindow | null = null
let db: ReturnType<typeof openDatabase>

function createWindow() {
  win = new BrowserWindow({
    width: 1366, height: 820, minWidth: 1100, minHeight: 650, show: false,
    title: 'Business Records',
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true, nodeIntegration: false, sandbox: true,
    },
  })
  win.once('ready-to-show', () => win?.show())
  win.setMenuBarVisibility(false)
  win.webContents.setWindowOpenHandler(() => ({ action: 'deny' }))

  if (isDev) win.loadURL('http://localhost:3000')
  else win.loadFile(path.join(app.getAppPath(), '.output', 'public', 'index.html'))
}

app.whenReady().then(() => {
  db = openDatabase(app.getPath('userData'))        // migrates and backs up first
  registerIpc(db, () => win)
  createWindow()
})

app.on('second-instance', () => { if (win) { win.restore(); win.focus() } })
app.on('before-quit', () => { runExitBackup(db); db.close() })
app.on('window-all-closed', () => app.quit())
```

electron/preload.ts

```
// electron/preload.ts
import { contextBridge, ipcRenderer } from 'electron'

const call = (channel: string) => (payload?: unknown) => ipcRenderer.invoke(channel, payload)

contextBridge.exposeInMainWorld('api', {
  auth: {
    status: call('auth:status'), setup: call('auth:setup'), login: call('auth:login'),
    logout: call('auth:logout'), changePassword: call('auth:changePassword'),
    recover: call('auth:recover'),
  },
  records: {
    list: call('records:list'), get: call('records:get'), create: call('records:create'),
    update: call('records:update'), setStatus: call('records:setStatus'),
    remove: call('records:delete'), restore: call('records:restore'), purge: call('records:purge'),
    duplicate: call('records:duplicate'), checkDuplicates: call('records:checkDuplicates'),
    suggestions: call('records:suggestions'),
  },
  statuses: {
    list: call('statuses:list'), create: call('statuses:create'), update: call('statuses:update'),
    reorder: call('statuses:reorder'), remove: call('statuses:delete'),
    merge: call('statuses:merge'),
  },
  settings: {
    get: call('settings:get'), set: call('settings:set'), chooseLogo: call('settings:chooseLogo'),
  },
  reports: {
    data: call('reports:data'), exportPdf: call('reports:exportPdf'),
    exportExcel: call('reports:exportExcel'),
  },
  backup: {
    now: call('backup:now'), list: call('backup:list'), restore: call('backup:restore'),
    chooseFolder: call('backup:chooseFolder'), testFolder: call('backup:testFolder'),
    check: call('backup:check'),
  },
  system: { openFolder: call('system:openFolder'), info: call('system:info') },
})
```

electron/ipc.ts

```
// electron/ipc.ts  (pattern used for every channel)
import { ipcMain } from 'electron'
import { z } from 'zod'
import { session } from './session'
import { AppError, toErrorResult, type Result } from './errors'

export function handle<S extends z.ZodTypeAny, R>(
  channel: string, schema: S, requireLogin: boolean,
  fn: (input: z.infer<S>) => R | Promise<R>,
) {
  ipcMain.handle(channel, async (_event, raw): Promise<Result<R>> => {
    try {
      if (requireLogin && !session.isLoggedIn()) {
        throw new AppError('NOT_LOGGED_IN', 'Please log in')
      }
      const input = schema.parse(raw ?? {})
      return { ok: true, data: await fn(input) }
    } catch (err) {
      return toErrorResult(err)              // friendly code + message, logged locally
    }
  })
}
```

electron/db/index.ts

```
// electron/db/index.ts
import Database from 'better-sqlite3'
import fs from 'node:fs'
import path from 'node:path'
import m001 from './migrations/001_initial.sql'   // tsup loader: '.sql' -> text
import { backupNow } from '../backup'
import { AppError } from '../errors'

const MIGRATIONS: string[] = [m001]               // index + 1 = schema version

export function openDatabase(userDataDir: string) {
  const file = path.join(userDataDir, 'data', 'records.db')
  fs.mkdirSync(path.dirname(file), { recursive: true })
  const db = new Database(file)
  db.pragma('foreign_keys = ON')
  db.pragma('synchronous = FULL')

  const current = db.pragma('user_version', { simple: true }) as number
  if (current > MIGRATIONS.length) {
    throw new AppError('DB_TOO_NEW', 'This data was made by a newer version of the program.')
  }
  if (current < MIGRATIONS.length) {
    if (current > 0) backupNow(db, 'before-migration')
    for (let v = current; v < MIGRATIONS.length; v++) {
      db.transaction(() => {
        db.exec(MIGRATIONS[v])
        db.pragma(`user_version = ${v + 1}`)
      })()
    }
  }
  return db
}
```

electron/db/repos/records.ts

```
// electron/db/repos/records.ts  (save pattern: one transaction per record)
const saveRecord = db.transaction((id: number | null, input: RecordInput, items: ItemInput[]) => {
  const { subtotal, tax, grandTotal } = totals(items, input.taxRateBp)
  const recordId = id === null
    ? insertRecord(input, subtotal, tax, grandTotal)
    : updateRecord(id, input, subtotal, tax, grandTotal)

  db.prepare('DELETE FROM record_items WHERE record_id = ?').run(recordId)
  items.forEach((it, i) => insertItem.run(
    recordId, i + 1, it.description, it.quantityX100, it.unitPriceMinor, lineTotal(it),
  ))
  writeAudit(id === null ? 'record.create' : 'record.update', 'record', recordId, input)
  return recordId
})
```

electron/auth.ts

```
// electron/auth.ts
import { scryptSync, randomBytes, timingSafeEqual } from 'node:crypto'

export function hashSecret(secret: string): string {
  const salt = randomBytes(16)
  const hash = scryptSync(secret.normalize('NFKC'), salt, 64)
  return `${salt.toString('hex')}:${hash.toString('hex')}`
}

export function verifySecret(secret: string, stored: string): boolean {
  const [saltHex, hashHex] = stored.split(':')
  const expected = Buffer.from(hashHex, 'hex')
  const salt = Buffer.from(saltHex, 'hex')
  const actual = scryptSync(secret.normalize('NFKC'), salt, expected.length)
  return timingSafeEqual(actual, expected)
}
```

electron/reports/pdf.ts

```
// electron/reports/pdf.ts
import { BrowserWindow } from 'electron'

export async function renderPdf(route: string, landscape = false): Promise<Buffer> {
  const w = new BrowserWindow({
    show: false,
    webPreferences: { preload: PRELOAD_PATH, contextIsolation: true, sandbox: true },
  })
  await loadAppRoute(w, route)                    // e.g. '/print/monthly?month=2026-10'
  await waitUntil(() => w.webContents.executeJavaScript('window.__reportReady === true'), 15000)

  const pdf = await w.webContents.printToPDF({
    pageSize: 'A4', landscape, printBackground: true,
    margins: { top: 0.6, bottom: 0.7, left: 0.5, right: 0.5 },          // inches
    displayHeaderFooter: true,
    headerTemplate: '<span></span>',
    footerTemplate: '<div style="font-size:8px;width:100%;text-align:center;color:#64748b">' +
      'Page <span class="pageNumber"></span> of <span class="totalPages"></span></div>',
  })
  w.destroy()
  return pdf
}
```

electron/reports/excel.ts

```
// electron/reports/excel.ts  (outline)
import ExcelJS from 'exceljs'

export async function buildWorkbook(data: ReportData, charts: Record<string, string>) {
  const wb = new ExcelJS.Workbook()
  const summary = wb.addWorksheet('Summary')
  const sheet = wb.addWorksheet('Records', { views: [{ state: 'frozen', ySplit: 1 }] })

  sheet.columns = [
    { header: 'Date', key: 'date', width: 12, style: { numFmt: 'dd/mm/yyyy' } },
    { header: 'Client', key: 'client', width: 28 },
    { header: 'Currency', key: 'currency', width: 10 },
    { header: 'Grand total', key: 'total', width: 16, style: { numFmt: '#,##0' } },
    // ...remaining columns
  ]
  sheet.getRow(1).font = { bold: true }
  sheet.autoFilter = { from: 'A1', to: { row: 1, column: sheet.columnCount } }
  data.records.forEach(r => sheet.addRow(r))

  // filter-aware total (SUBTOTAL 109 ignores rows hidden by the filter)
  const last = sheet.rowCount
  sheet.addRow({ client: 'Total', total: { formula: `SUBTOTAL(109,D2:D${last})` } })

  for (const [name, png] of Object.entries(charts)) {            // chart images from Chart.js
    const id = wb.addImage({ base64: png, extension: 'png' })
    summary.addImage(id, { tl: { col: 8, row: 1 }, ext: { width: 480, height: 280 } })
  }
  return wb.xlsx.writeBuffer()
}
```

Appendix D. Build and packaging files

Create the project

```
npm create nuxt@latest business-records          (choose the Nuxt 4 minimal template)
cd business-records
npm i pinia @pinia/nuxt @nuxtjs/i18n zod better-sqlite3 exceljs chart.js vue-chartjs
npm i @fontsource-variable/inter lucide-vue-next date-fns
npm i -D electron electron-builder tsup concurrently wait-on typescript vitest
npm i -D tailwindcss @tailwindcss/vite @types/better-sqlite3
```

package.json (scripts)

```
{
  "name": "business-records",
  "version": "1.0.0",
  "main": "dist-electron/main.js",
  "scripts": {
    "postinstall": "electron-builder install-app-deps",
    "dev": "concurrently -k \"nuxt dev\" \"npm:dev:electron\"",
    "dev:electron": "wait-on tcp:3000 && tsup --watch --onSuccess \"electron .\"",
    "build:web": "nuxt generate",
    "build:electron": "tsup",
    "build": "npm run build:web && npm run build:electron",
    "dist": "npm run build && electron-builder --win --x64",
    "test": "vitest run"
  }
}
```

tsup.config.ts

```
// tsup.config.ts
import { defineConfig } from 'tsup'

export default defineConfig({
  entry: ['electron/main.ts', 'electron/preload.ts'],
  outDir: 'dist-electron',
  format: ['cjs'],
  target: 'node20',
  external: ['electron', 'better-sqlite3'],
  loader: { '.sql': 'text' },
  clean: true,
})
```

electron-builder.yml

```
# electron-builder.yml
appId: com.example.businessrecords
productName: Business Records
artifactName: BusinessRecords-Setup-${version}.exe
directories:
  output: release
files:
  - dist-electron/**
  - .output/public/**
  - package.json
asarUnpack:
  - "**/node_modules/better-sqlite3/**"
npmRebuild: true
win:
  target:
    - target: nsis
      arch: [x64]
  icon: build/icon.ico
nsis:
  oneClick: false
  perMachine: false
  allowToChangeInstallationDirectory: true
  createDesktopShortcut: always
  createStartMenuShortcut: true
  shortcutName: Business Records
  deleteAppDataOnUninstall: false
```

Build steps on the Windows build computer: `npm install`, then `npm run dist`. The installer appears in the `release` folder. Change the appId and the product name to the final names before the first release, because the appId decides where Windows keeps the program's data folder.

Appendix E. Report queries

All report figures come from plain SQL. Parameters are shown with a colon. Records in the recycle bin (`deleted_at` not empty) are always excluded.

electron/reports/queries.ts (SQL parts)

```
-- Daily summary: counts and value per status and currency for one day
SELECT s.name AS status, s.color, r.currency,
       COUNT(*) AS records,
       SUM(CASE WHEN s.exclude_from_totals = 0 THEN r.grand_total_minor ELSE 0 END) AS total_minor
FROM records r JOIN statuses s ON s.id = r.status_id
WHERE r.deleted_at IS NULL AND r.record_date = :day
GROUP BY s.id, r.currency
ORDER BY s.sort_order;

-- Monthly summary: per-day breakdown (:start = first day, :end = first day of next month)
SELECT r.record_date, r.currency, COUNT(*) AS records,
       SUM(CASE WHEN s.exclude_from_totals = 0 THEN r.grand_total_minor ELSE 0 END) AS total_minor
FROM records r JOIN statuses s ON s.id = r.status_id
WHERE r.deleted_at IS NULL AND r.record_date >= :start AND r.record_date < :end
GROUP BY r.record_date, r.currency
ORDER BY r.record_date;

-- Top 10 clients by value in the period
SELECT r.client_name, r.currency, COUNT(*) AS records,
       SUM(CASE WHEN s.exclude_from_totals = 0 THEN r.grand_total_minor ELSE 0 END) AS total_minor
FROM records r JOIN statuses s ON s.id = r.status_id
WHERE r.deleted_at IS NULL AND r.record_date >= :start AND r.record_date < :end
GROUP BY r.client_name COLLATE NOCASE, r.currency
ORDER BY total_minor DESC
LIMIT 10;

-- Source of information
SELECT COALESCE(NULLIF(TRIM(r.source_of_info), ''), '(none)') AS source, COUNT(*) AS records
FROM records r
WHERE r.deleted_at IS NULL AND r.record_date >= :start AND r.record_date < :end
GROUP BY source COLLATE NOCASE
ORDER BY records DESC;

-- Upcoming events
SELECT r.event_date, r.client_name, r.location, r.contacts, s.name AS status, r.grand_total_minor
FROM records r JOIN statuses s ON s.id = r.status_id
WHERE r.deleted_at IS NULL AND r.event_date BETWEEN :from AND :to
ORDER BY r.event_date, r.client_name;

-- Duplicate number check (before saving)
SELECT id, record_date, client_name FROM records
WHERE deleted_at IS NULL AND invoice_no = :value AND id <> :excludeId;
```